



UNIVERSIDAD CATÓLICA ANDRÉS BELLO
FACULTAD DE INGENIERÍA
INGENIERÍA INFORMÁTICA
ORGANIZACIÓN DEL COMPUTADOR
NRC: 25765



Informe:

Proyecto Ensamblador

Prof.: Omar Hernández

Integrantes: Veronica Betancourt

Sara Rojas

Nicola Sigillo

Gabriela Vasquez

Caracas, julio 2026.

INTRODUCCIÓN

El presente informe detalla el diseño, la estructura y el análisis del proyecto en lenguaje ensamblador desarrollado por el equipo. En el mismo, manipulamos matrices al realizar diversas operaciones propias de estas: como la transposición, la suma, o la negación. Al trabajar en el nivel de la arquitectura de 16 bits (definido con .MODEL SMALL, al inicio del proyecto), una matriz no es más que un bloque contiguo de memoria RAM, y operar con ella implica calcular los desplazamientos utilizando registros de propósito general y modos de direccionamiento usando índices.

El objetivo principal de este software consiste en que, a partir de una matriz A de 8x8, se calcule una matriz B: definida como la negación de la traspuesta de A; realicemos la suma de ambas matrices A y B, y finalmente sumemos la matriz A con la traspuesta de B. Adicionalmente, para comprobar que la operación fue realizada correctamente, se imprimirán los resultados a través de un formato visual limpio y controlado por interrupciones del sistema operativo (usando INT 21H).

DESARROLLO

Organización de Datos y Modos de Acceso a Memoria

En la sección .DATA, definimos el índice máximo de las matrices mediante la variable N DW 7, el cual representa el índice máximo de la matriz definida (en este caso, una 8x8, dado a que los índices en ensamblador se trabajan a partir del 0). Para su lectura en memoria, adicionalmente cargamos el valor de N en el registro de puntero base (BP) y lo incrementamos en 1 (*INC BP*), de forma que pudiera realizar las iteraciones de la manera correcta.

A partir de este punto, utilizamos BP (que ahora contiene la dimensión física real de la matriz) como contador en los ciclos y multiplicador matemático. La fórmula general para acceder a un elemento en orden de filas y calcular su posición relativa (el desplazamiento u offset) respecto al inicio del segmento de datos, se definió como:

$$\text{offset} = (i * BP + j) * 2$$

donde multiplicamos la fila i por el número de columnas BP , le sumamos la columna j para obtener el índice lineal, y multiplicamos por 2 porque cada palabra (word) consume 2 bytes. En el código, optimizamos esta multiplicación por 2 utilizando la instrucción de desplazamiento de bits *SHL AX, 1* en lugar de una multiplicación genérica, tomamos esta decisión porque, a nivel de hardware que reduce los ciclos de reloj necesarios para evaluar la dirección efectiva almacenada en el registro índice SI .

Algoritmos de Transposición y Negación para la Matriz B

Para calcular la matriz B bajo la regla lógica $B[i, j] = -A[j, i]$, implementamos un algoritmo de doble bucle anidado. El iterador exterior gestiona las filas de B mediante el registro de conteo CX (respaldado en la pila con *PUSH CX*) y el índice de fila BX (inicializado con *XOR BX, BX*). El bucle interior gestiona las columnas utilizando el registro DI (*XOR DI, DI*).

Para evaluar la posición $A[j, i]$ (A traspuesta), invertimos los roles de los índices operando primero con la columna DI multiplicada por BP y sumándole la fila BX . Esto genera el índice lineal traspuesto para extraer el valor con el modo de direccionamiento indexado $A[SI]$. De esta manera también traspusimos B , llegado el momento. En cuanto a la negación, simplemente aplicamos la instrucción *NEG AX* para invertir el signo del valor entero con signo.

Finalmente, para almacenar el cálculo de la negación de la traspuesta de A y guardarlo en B , calculamos el offset de destino en orden normal de filas para $B[i, j]$ multiplicando la fila BX por BP y sumándole la columna DI . De esta manera, el resultado negativo se almacena en memoria mediante *MOV B[SI], AX*.

Para imprimir las matrices $A + B$, aprovechamos que ambas estructuras comparten idéntica disposición lineal en memoria. Aquí empleamos un recorrido secuencial directo utilizando un único índice lineal SI que avanza de 2 en 2 bytes (*ADD SI, 2*), simplificando el costo computacional del algoritmo de recorrido.

Subrutinas de Apoyo

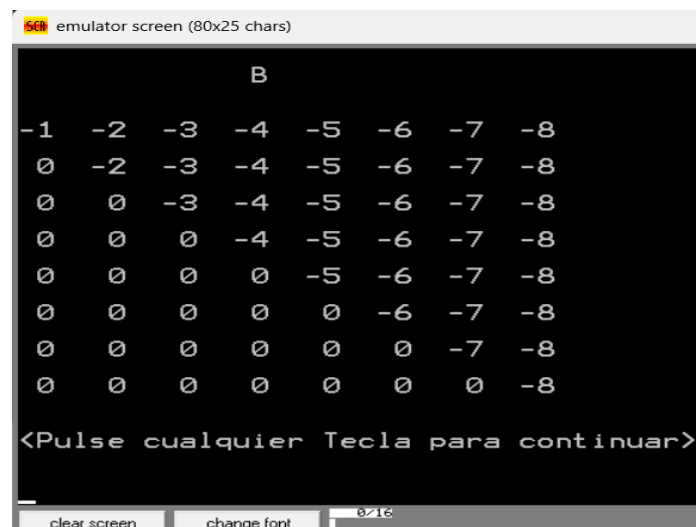
- **PRINT_NUM:** Esta subrutina gestiona la impresión de números enteros con signo de manera pulcra. Verifica si el número es negativo evaluando *OR AX, AX* y luego procede a un salto condicional con *JNS*. Si es negativo, imprime directamente el carácter '-' mediante la función 02H de la interrupción INT 21H y convierte el número a positivo con *NEG*. En cambio, si es positivo, inyecta un espacio en blanco para lograr una alineación simétrica perfecta respecto a los números que poseen el signo menos cuando vaya a imprimirse toda la matriz.

Luego, realiza divisiones sucesivas entre 10 (*MOV BX, 10*), apilando los residuos y desapilándolos en orden inverso para convertirlos a su representación ASCII en sistema decimal.

- **PAUSA:** Diseñada para pausar la ejecución del programa llamando al mensaje *MsgPausa* y utilizando la función de lectura de teclado sin eco 07H, es decir sin que se muestre la tecla que el usuario presionó, de la interrupción INT 21H, permitiendo al usuario controlar el ritmo de visualización de cada matriz en la consola del emulador antes de avanzar a la siguiente sección.

Salidas en Consola

Además de lo anteriormente mencionado, para la impresión en consola se imprimieron primero los encabezados (*MsgB*, *MsgAB*, *MsgATB*, según fuera el caso) y se hacía uso del salto de línea *NuevaL* para una mejor visualización.



The screenshot shows an emulator window titled "emulator screen (80x25 chars)". The main display area is black with white text. At the top, the letter "B" is centered. Below it is an 8x8 matrix of numbers. The first row contains the numbers -1 through -8. The subsequent seven rows start with a '0' followed by a space, and then contain numbers from -2 to -8, with the first column of these rows being empty. At the bottom of the matrix, the text "<Pulse cualquier Tecla para continuar>" is displayed. Below the main display area, there is a status bar with three buttons: "clear screen", "change font", and a small indicator showing "0/16".

```
emulator screen (80x25 chars)

      B
-1  -2  -3  -4  -5  -6  -7  -8
0   -2  -3  -4  -5  -6  -7  -8
0   0   -3  -4  -5  -6  -7  -8
0   0   0   -4  -5  -6  -7  -8
0   0   0   0   -5  -6  -7  -8
0   0   0   0   0   -6  -7  -8
0   0   0   0   0   0   -7  -8
0   0   0   0   0   0   0   -8

<Pulse cualquier Tecla para continuar>

clear screen  change font  0/16
```

```
emulator screen (80x25 chars)

      A + B
0  -2  -3  -4  -5  -6  -7  -8
2   0  -3  -4  -5  -6  -7  -8
3   3   0  -4  -5  -6  -7  -8
4   4   4   0  -5  -6  -7  -8
5   5   5   5   0  -6  -7  -8
6   6   6   6   6   0  -7  -8
7   7   7   7   7   7   0  -8
8   8   8   8   8   8   8   0

<Pulse cualquier Tecla para continuar>
```

```
emulator screen (80x25 chars)

      A + B (traspuesto)
0   0   0   0   0   0   0   0
0   0   0   0   0   0   0   0
0   0   0   0   0   0   0   0
0   0   0   0   0   0   0   0
0   0   0   0   0   0   0   0
0   0   0   0   0   0   0   0
0   0   0   0   0   0   0   0
0   0   0   0   0   0   0   0

Fin del programa.
```

CONCLUSIONES

El desarrollo de este proyecto en ensamblador nos permitió desarrollar mayor entendimiento del lenguaje orientado al hardware a nivel de máquina. La correcta aplicación de fórmulas de direccionamiento lineal para arreglos bidimensionales, la optimización mediante corrimientos de bits (SHL), y el manejo riguroso de la pila para evitar la colisión de registros durante operaciones críticas, garantizan que el código no solo sea matemáticamente correcto, sino robusto y eficiente en tiempo de ejecución.)

Adicionalmente, la implementación del registro BP para calcular dimensiones reales a partir de índices máximos en tiempo de ejecución demostró la flexibilidad del ensamblador para adaptar fórmulas matemáticas abstractas sin comprometer la disposición física de los datos en la memoria.

REFERENCIAS

Irvine, K. R. (2019). *Lenguaje Ensamblador para x86*. (8va ed.). Pearson.

Xmdi. (2021, noviembre 5). *Python vs C/C++ vs Assembly side-by-side comparison*
[Video]. YouTube. <https://www.youtube.com/watch?v=3PclJKd1PKU>